



Metoda Greedy

Problema spectacolelor · Diverse probleme

Andrei Mihai Ciobanu

Algolymp Summer School

Ce este Greedy?

Problema spectacolelor

Alte aplicații

Recapitulare

Ce este Greedy?

Problemă

Se dau monede de valori 1, 5, 10, 50 lei. Trebuie dat rest de R lei folosind **cât mai puține monede**.

Strategie naturală: la fiecare pas, se alege **cea mai mare monedă** care nu depășește restul rămas.

- Nu se ține cont de pașii următori — doar de cel mai bun pas curent
- Odată aleasă o monedă, nu se mai revine
- Soluția se construiește **pas cu pas**, fără a ne schimba deciziile pe parcurs

Aceasta este esența gândirii **greedy**.

Motivație — Rezolvare pentru $R = 63$

Rest rămas	Monedă aleasă	Observație
63	50	cea mai mare monedă ≤ 63
13	10	cea mai mare monedă ≤ 13
3	1	$5 > 3$, deci se alege 1
2	1	
1	1	
0	—	total: 5 monede

La fiecare pas s-a ales **varianta locală optimă**. Nu se știa de la început că vor fi necesare 5 monede — soluția s-a construit pas cu pas.

Algoritm Greedy

Un algoritm **greedy** construiește soluția pas cu pas, făcând la fiecare pas **cea mai bună alegere locală**, fără a reveni ulterior asupra deciziilor luate.

- **Fără backtracking** — fiecare decizie e finală
- **Eficient** — complexitate tipică $O(n \log n)$ sau $O(n)$
- **Simplu de implementat** — de obicei sortare + parcurgere

Greedy $\neq$ algoritm specific

Greedy este o **paradigmă** — o strategie de gândire. Alegerea “locală optimă” diferă de la problemă la problemă. **Nu garantează soluția optimă pentru orice problemă.**

Structura tipică

Majoritatea algoritmilor greedy urmează același tipar:

1. **Identifică criteriul de ordonare**
2. **Sortează** elementele după acel criteriu
3. **Parcurge și alege greedy** la fiecare pas
4. **Verifică corectitudinea** alegerii

Pro tip

La concursuri se poate sări direct la submit — “*Proof by AC*” (USACO Guide).

Când NU funcționează Greedy?

Contraexemplu — monede nestandard $\{1, 3, 4\}$, rest = 6:

Greedy (cea mai mare monedă):

- Alege 4 rest = 2
- Alege 1 rest = 1
- Alege 1 rest = 0

⇒ **3 monede** suboptim

Soluție optimă:

- Alege 3 rest = 3
- Alege 3 rest = 0

⇒ **2 monede** ✓

Concluzie

Greedy poate da soluții **incorecte**. Trebuie să verificăm că alegerea locală optimă conduce la cea globală optimă.

Problema spectacolelor

Problema spectacolelor

Se dau n spectacole cu **start** s_i și **final** f_i . Câte spectacole poți viziona **cel mult**, știind că nu pot rula două simultan?

Exemplu ($n = 5$):

Spectacol	Start	Final
Film A	9	11
Film B	12	14
Film C	15	17
Film D	9	15
Film E	13	18

Răspuns: 3 spectacole (A, B, C).

De ce nu sortăm după start?

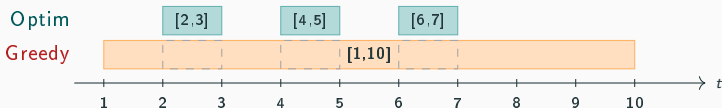
Contraexemplu: activități $[1, 10]$, $[2, 3]$, $[4, 5]$, $[6, 7]$

Greedy după start (crescător)

Ordine: $[1, 10]$, $[2, 3]$, $[4, 5]$, $[6, 7]$

- Alege $[1, 10]$ ($last = 10$)
- $[2, 3]$: $2 < 10$ — sare
- $[4, 5]$: $4 < 10$ — sare
- $[6, 7]$: $6 < 10$ — sare

Rezultat: **1 activitate**



De ce nu sortăm după durată?

Contraexemplu: activități $[1, 5]$, $[4, 6]$, $[5, 9]$

Greedy după durată (crescătoare: 2, 4, 4)

Ordine: $[4, 6]$, $[1, 5]$, $[5, 9]$

- Alege $[4, 6]$ ($last = 6$)
- $[1, 5]$: $1 < 6$ — sare
- $[5, 9]$: $5 < 6$ — sare

Rezultat: **1 activitate**



Criteriu

La fiecare pas, alege spectacolul care **se termină cel mai devreme** și nu se suprapune cu ultimul ales.

De ce funcționează?

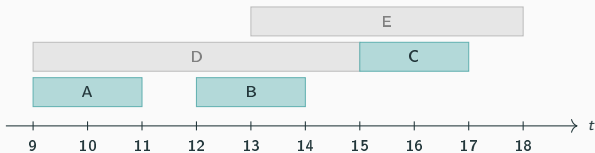
- Un spectacol care se termină mai devreme lasă **mai mult timp liber** pentru spectacolele viitoare
- Orice altă alegere (cu final mai târziu) lasă **cel mult la fel de mult timp liber** — niciodată mai mult
- Prin urmare, alegerea cu finalul cel mai mic este **întotdeauna la fel de bună sau mai bună**

Algoritm — Problema spectacolelor

1. Sortează spectacolele după f_i crescător
2. Inițializează: $cnt \leftarrow 0$, $last \leftarrow -\infty$
3. **Pentru fiecare** spectacol (în ordine):
 dacă $s_i \geq last$: alege-l ($cnt++$, $last \leftarrow f_i$)

Parcurgerea este **liniară** — fiecare spectacol e inspectat o singură dată. Costul dominant este sortarea.

Exemplu pas cu pas



Spectacol	s_i	f_i	Decizie
Film A	9	11	$9 \geq -\infty \checkmark$ ales (<i>last</i> = 11)
Film B	12	14	$12 \geq 11 \checkmark$ ales (<i>last</i> = 14)
Film D	9	15	$9 < 14 \times$ sărit
Film C	15	17	$15 \geq 14 \checkmark$ ales (<i>last</i> = 17)
Film E	13	18	$13 < 17 \times$ sărit

Răspuns: $cnt = 3$ — Film A, Film B, Film C.

Implementare

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Spectacol {
5      int start, finish;
6  };
7
8  bool cmp(Spectacol a, Spectacol b) {
9      return a.finish < b.finish;
10 }
```

Comparatorul cmp

Sortăm după finish (ora de final) crescător — exact criteriul greedy.

Implementare

```
1  int main() {
2      int n; cin >> n;
3      vector<Spectacol> v(n);
4      for (int i = 0; i < n; i++)
5          cin >> v[i].start >> v[i].finish;
6
7      sort(v.begin(), v.end(), cmp);
8
9      int cnt = 0, last = -1;
10     for (int i = 0; i < n; i++) {
11         if (v[i].start >= last) {
12             cnt++;
13             last = v[i].finish;
14         }
15     }
16     cout << cnt << "\n";
17 }
```

Pas	Operație	Complexitate
Citire	n spectacole	$O(n)$
Sortare	după ora de final	$O(n \log n)$
Parcurgere	o singură trecere	$O(n)$
Total		$O(n \log n)$

Spațiu și corectitudine

Spațiu: $O(n)$. Corectitudine: **exchange argument** — orice soluție optimă poate fi transformată în alegerea greedy fără a pierde activități.

Alte aplicații

Selectarea celor mai profitabile k elemente

Problemă

Dintr-un tablou de n valori, alege k elemente astfel încât **suma lor să fie maximă**.

Strategie: sortează descrescător $\rightarrow$ ia primele k elemente.

Exemplu: $a = [3, 7, 1, 9, 4]$, $k = 3$

- Sortat: $[9, 7, 4, 3, 1]$
- Suma: $9 + 7 + 4 = 20$ (optim)

Corectitudine: înlocuind orice element din top k cu unul mai mic, suma ar **scădea**. **Complexitate:** $O(n \log n)$

Problemă

Se dau m intervale $[l_i, r_i]$. Acoperă segmentul $[0, L]$ complet folosind **cât mai puține intervale**.

Strategie greedy

Fie pos = capătul drept acoperit până acum (inițial 0). La fiecare pas:

1. Consideră toate intervalele cu $l_i \leq pos$
2. Alege cel cu r_i **maxim**
3. Actualizează $pos \leftarrow r_i$

Repetă până când $pos \geq L$.

Acoperire cu intervale — Exemplu

$L = 10$, intervale: $[0, 4]$, $[1, 7]$, $[2, 5]$, $[5, 10]$, $[6, 12]$

Pas	pos	Eligibile ($l \leq pos$)	Ales
1	0	$[0, 4]$	$[0, 4] \rightarrow pos = 4$
2	4	$[1, 7]$, $[2, 5]$	$[1, 7] (\max r) \rightarrow pos = 7$
3	7	$[5, 10]$, $[6, 12]$	$[6, 12] (\max r) \rightarrow pos = 12$

$pos = 12 \geq 10 = L$ — acoperit complet. **Răspuns:** 3 intervale: $[0, 4]$, $[1, 7]$, $[6, 12]$.

Greedy nu este mereu corect!

Trebuie să argumentăm că alegerea locală optimă conduce la soluția globală optimă.

Cum verificăm corectitudinea?

1. **Exchange argument** — presupui că există o soluție mai bună și arăți că o poți transforma în alegerea greedy fără a pierde calitate
2. **Testare empirică** — compară cu brute force pe cazuri mici ($n \leq 15$)
3. ***Proof by AC*** — trimiți direct la judge și verifici (metodă comună la concursuri)

Recapitulare

Recapitulare

Problemă	Criteriu greedy	Complexitate
Spectacole	Sortare după final	$O(n \log n)$
k elemente maxime	Sortare descrescătoare	$O(n \log n)$
Monede standard	Cea mai mare $\leq$ rest	$O(n)$
Acoperire intervale	Extindere maximă	$O(n \log n)$

Pași tipici:

1. Identifică **criteriul de ordonare**
2. Sortează + parcurge **liniar**
3. Verifică **corectitudinea** (exchange argument / empiric)

Probleme de antrenament

1. **Increasing Array** (CSES #1094)
2. **Card Game for Two** (AtCoder ABC088 B)
3. **Apartments** (CSES #1084)
4. **Ferris Wheel** (CSES #1090)
5. **Movie Festival** (CSES #1629)
6. **Tasks and Deadlines** (CSES #1630)

1. **pbinfo** — Metoda Greedy (în română)
2. **USACO Guide** — Introduction to Greedy Algorithms (Bronze)
3. **USACO Guide** — Greedy Algorithms with Sorting (Silver)
4. **CPH** — cap. 6 — Greedy Algorithms